ARTIFICIAL INTELLIGENCE

(Real-World Problem Solving Using Artificial Intelligence Approaches)

Project on:

“Plant Species Classification“

Submitted By

Cem Karasu (s5375415)

Submitted On

15-05-2025

Table of Content

Abstract.....3
1. Introduction.....4
2. The Real-World Problem.....4
3. Project Aim and Objectives.....4
4. Adopted Artificial Intelligence Approach.....5
5. Artificial Intelligence Approach Implementation.....6
6. Evaluation, Results and Discussions.....11
7. Conclusion and Future Work.....13
References.....14
Appendix.....14

Table of Figures

Figure 1 Architecture of Custom CNN 7
Figure 2 Fully Connected Layer Architecture of VGG16 9
Figure 3 Fully Connected Layer Architecture of ResNet50.....9
Figure 4 Fully Connected Layer Architecture of MobileNetV3.....10
Figure 5 Fully Connected Layer Architecture of EfficientNetB0 10
Figure 6 Five Model Result Summary 11
Figure 7 Model Comparison Graph 12
Figure 8 Six Performance Comparison Graphs 12

Abstract

This project addresses the problem of classifying plant species using artificial intelligence. Accurate identification plays a vital role in fields such as agriculture, farming, and camping. Because most plant species are difficult for humans to distinguish, manual classification is time-consuming and error-prone.

To overcome this problem, five different machine learning models were created. A combined dataset with approximately 300,000 images and 226 different plant species is used to train the models. Deep learning techniques, such as data augmentation. dropout, L2 regularisation, and learning rate scheduler were used to prevent overfitting and make the model more generalised.

Models were evaluated using appropriate performance metrics: accuracy, loss, precision, recall, and F1-score. This study shows how artificial intelligence can effectively solve real-world agricultural problems and provides the basis for plant recognition systems.

1. Introduction

Classifying plant species is crucial for agriculture, botany, camping, and nature-related activities. It helps identify invasive species and differentiate useful plants from harmful ones. However, the vast number of species and their similarities make this task challenging for humans.

This project focuses on 226 plant species, including fruits, vegetables, and flowers, with 200,000 images. Traditional methods struggle with this data volume, but artificial intelligence enables accurate classification and management of such extensive datasets.

2. The Real-World Problem

There are hundreds of thousands of plant species in the world. These species have a vital role directly or indirectly in many areas such as agriculture, camping, and botany. For a human being, it is impossible to identify these species, specifically for non-experts. Similarities between plants in terms of colour patterns and shape, different development stages of the same species, and seasonal changes cause manual classification to be both time-consuming and prone to errors.

Moreover, the demand for plant identification is not limited only to academic or agricultural studies and research. Farmers need to identify the plants they encounter in their daily work to separate harmful species from beneficial species. By accurately identifying harmful species, they can increase production on their farm, which in turn can increase their revenue. Gardeners or anyone who spends their time gardening can efficiently care for, grow, or collect the plants they encounter in nature. Also, campers and nature hikers should know if the plants they encounter are edible, medicinal or poisonous. And all of this shows great importance in the fields of health and safety.

In addition to these, global environmental events like climate change may change the habitats of some plant species, resulting in them being found in unusual regions. This causes traditional methods to be inefficient.

3. Project Aim and Objectives

The main goal of this project is to develop an accurate and reliable plant species classification system using a large and well-structured plant species dataset. To achieve this objective, the objectives that need to be completed are listed below:

- Preparing the dataset

- Developing a Custom CNN Model
- Developing other CNN models using transfer learning
- Performance Analysis
- Evaluate and Compare

4. Adopted Artificial Intelligence Approach

To solve the problem of accurately classifying plant species, different machine learning methods are researched to find the most suitable technique to create a model.

First, classical machine learning methods such as Support Vector Machines (SVM), k-Nearest Neighbours (k-NN) and Decision Trees were investigated. Wang et al., 2021 showed that these classical methods cannot perform sufficiently enough in high dimensional and complex data such as images. Their paper compares traditional methods (SVM) and deep learning methods (CNN). They observed that when dimensionality increases, SVM performance drops compared to CNN.

Karypidis et al., 2022, also compared traditional methods (k-NN and SVM) and Deep learning methods (CNN). Their paper shows that the traditional model's performance decreases when the feature size increases. It also indicates that CNN outperforms traditional models in image classification. Since traditional methods are insufficient in image classification, they are not considered in this project.

Next, a Deep learning method, CNN, are investigated. AlexNet 's remarkable success in the ImageNet competition 2012 showed that CNNSs have revolutionised visual classification problems (Krizhevsky et al., 2012). This revolution led to new deep learning architectures being proposed. In the following studies, deep learning architectures such as VGGNet (Simonyan & Zisserman, 2015), ResNet (He et al., 2016) and MobileNet (Howard et al., 2017) have been introduced. And they show that by creating deeper, more compact and more efficient CNN architectures, model accuracy can be increased remarkably. The performance of the CNN in high-dimensional data, feature extraction, pattern recognition and classification make CNN the first choice for the visual classification tasks. Its ability to automatically learn hierarchical representation reduces the need for manual feature engineering.

Furthermore, the idea of transfer learning has been recognized as an efficient approach, especially with small to medium-sized datasets (Pan & Yang, 2010). Yosinski et al. (2014) demonstrated that the features acquired, particularly in the initial layers of pre-trained networks, can be reused for various tasks, leading to quicker learning and improved generalization.

In conclusion, in this project, two artificial intelligence methods were used:

- A custom CNN design trained from scratch
- Four models are implemented using transfer learning on four different architectures: VGG16. ResNet50, MobileNetV3 and EfficientNetB0.

5. Artificial Intelligence Approach Implementation

If a well-designed deep learning model is trained on a sufficiently large and diverse dataset of labelled plant images, it can accurately classify plant species. Also, using transfer learning from pre-trained convolutional neural networks can improve performance and training efficiency, making the model more suitable for real-world deployment.

In this project, five different CNN models were trained using a CNN trained from scratch and four different transfer learning models trained on top of the pre-trained weights. PyTorch, a Python library, is used to develop models. In addition, some helper functions from sklearn, tqdm, and matplotlib were also used in this project.

The dataset is a combination of the Vegetable Image Dataset (Ahmed and Mamun, 2021), the Oxford 102 Flower Dataset (Mohamed, 2024), the Ornamental Plant Images Dataset (Arfirza, n.d.), Fruit-360 (Oltean, 2017) and the Plant Type Datasets (Sulistya, 2023). It has 226 classes and approximately 200,000 images. The dataset was already split into Train and Test set folders. The separation ratio is 70% train, 30% test. However, it combines different datasets; therefore, separation is not balanced across the classes. To balance it, sklearn's train_test_split function is used to split the dataset into 70% train, 15% validation and 15% test. At the end, the train set has 123796 images. The test and validation sets have 26528 images each.

Total Number of Samples	Train Set	Test Set	Validation Set
176852	123796	26528	26528

Model one - Custom CNN:

- A CNN was created with three convolution layers (Conv2d), two fully connected layers (FC), max pooling, Relu, and dropout.

```
class CNN(nn.Module):
    def __init__(self,num_classes):
        super(CNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.conv3 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.3)
        self.fc1 = nn.Linear(64 * 12 * 12, 1024)
        self.fc2 = nn.Linear(1024, 512)
        self.fc3 = nn.Linear(512, num_classes)

    def forward(self, x):
        x = self.conv1(x)
        x = self.relu(x)
        x = self.pool(x)

        x = self.conv2(x)
        x = self.relu(x)
        x = self.pool(x)

        x = self.conv3(x)
        x = self.relu(x)
        x = self.pool(x)

        x = x.view(x.size(0), -1)

        x = self.fc1(x)
        x = self.relu(x)
        x = self.dropout(x)

        x = self.fc2(x)
        x = self.relu(x)
        x = self.dropout(x)

        x = self.fc3(x)

        return x
```

Figure 1 Architecture of Custom CNN

- To reduce the chance of overfitting, the following data augmentation methods are used:
    - Random affix
    - Random horizontal flip
    - Random vertical flip

    - Color Jitter
    - Center Crop

    By applying this transformation randomly to the image, each epoch model has a chance to train on an augmented image. Therefore, this technique can solve overfitting caused by class imbalance.

- In addition, by using dropout and L2 regularisation, the model was prevented from memorising training data.
    - Dropout randomly disables a fraction of the neurons during training. The fraction used in the training was 0.3 (30%).
    - L2 regularisation penalised large weights in the network, improving models' generalisation.

- Adam Optimiser is used because it adjusts the learning rate automatically using momentum. This helps the model learn faster and more accurately. The learning rate was 0.0001 for the training.

- Even though Adam Optimiser adjusts the learning rate using momentum, it can get stuck in the local minima. ReduceROonPlateaue solves this problem by lowering the learning rate to a specified amount when the model gets stuck.

Model Two – VGG16 (Transfer Learning)

- The VGG16 architecture pre-trained on ImageNet was used.

- All the convolutional layers were frozen to preserve the learned low-level features.

- The feature extraction layers were frozen; custom fully connected layers were added for classification:

```
model.classifier = nn.Sequential(
    nn.Linear(25088, 4096),
    nn.ReLU(),
    nn.Dropout(0.5),
    nn.Linear(4096, 1024),
    nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(1024, len(dataset_train.classes))
)
```

Figure 2 Fully Connected Layer Architecture of VGG16

- Overfitting was reduced by:
    - Dropout (0.4) and L2 regularisation
    - Training used Adam Optimiser with a learning rate of 0.0005.

Model Three – ResNet50 (Transfer Learning)

- ResNet50 architecture, pre-trained on ImageNet, was used with pre-trained ImageNet weights.

- The feature extraction layers were frozen; custom fully connected layers were added for classification:

```
model.fc = nn.Sequential(
    nn.Linear(model.fc.in_features, 1024),
    nn.ReLU(),
    nn.Dropout(0.4),
    nn.Linear(1024, 226)
)
```

Figure 3 Fully Connected Layer Architecture of ResNet50

- Overfitting was reduced by:
    - Dropout (0.4) and L2 regularisation
    - Training used Adam Optimiser with a learning rate of 0.0005.

Model Four – MobileNetV3 (Transfer Learning)

- MobileNetV3, known for its lightweight architecture, was used with pre-trained ImageNet weights.

- The feature extraction layers were frozen; custom fully connected layers were added for classification:

```
model.classifier = nn.Sequential(
    nn.Linear(model.classifier[0].in_features, 1024),
    nn.ReLU(),
    nn.Dropout(0.4),
    nn.Linear(1024, 226)
)
```

Figure 4 Fully Connected Layer Architecture of MobileNetV3

- Overfitting was reduced by:
    - Dropout (0.3) and L2 regularisation.
    - Optimiser: Adam, learning rate of 0.0005.

Model Five – EfficientNetB0 (Transfer Learning)

- EfficientNetB0 was selected for its compound scaling method, which balances network width, depth, and resolution.

- The feature extraction layers were frozen; custom fully connected layers were added for classification:

```
model.classifier = nn.Sequential(
    nn.Linear(model.classifier[1].in_features, 1024),
    nn.ReLU(),
    nn.Dropout(0.4),
    nn.Linear(1024, 226)
)
```

Figure 5 Fully Connected Layer Architecture of EfficientNetB0

- Overfitting was reduced by:
    - Dropout (0.4) and L2 regularisation
    - Training used Adam Optimiser with a learning rate of 0.0005.

A learning rate of 0.0001 was used for the custom CNN to allow stable training from scratch. For transfer learning models, a 0.0005 learning rate provides faster fine-tuning of pretrained weights without disrupting learned features.

In the models expect MobileNet, the dropout was set to 0.4 to reduce overfitting. For MobileNet, 0.3 was used since it already has a lightweight architecture.

At the end of the epoch, the validation set measures model performance for all models. It helps the model to stop early when it won't improve. If the validation loss decreases from the previous epoch, the best model is saved as pth.

During the training and testing, train losses, train accuracies, validation losses, validation accuracies, test loss, test accuracy and training time were recorded to identify the best model.

6. Evaluation, Results and Discussions

Five models were compared with each other using the obtained results. Metrics used to create performance results are accuracy, loss, precision, f1-score, recall and training time.

Model Training Summary

Model	Time (min)	Train Acc	Test Acc	Train Loss	Test Loss
Custom CNN	332.5782	85.1223%	86.8177%	0.5448	0.4922
VGG16	55.8268	97.8982%	96.4340%	0.0652	0.1235
ResNet50	91.9646	98.6001%	98.0926%	0.0500	0.0621
MobileNet	33.7176	99.1688%	98.3037%	0.0393	0.0618
EfficientNet	53.0481	98.6058%	98.1868%	0.0602	0.0704

Figure 6 Five Model Result Summary

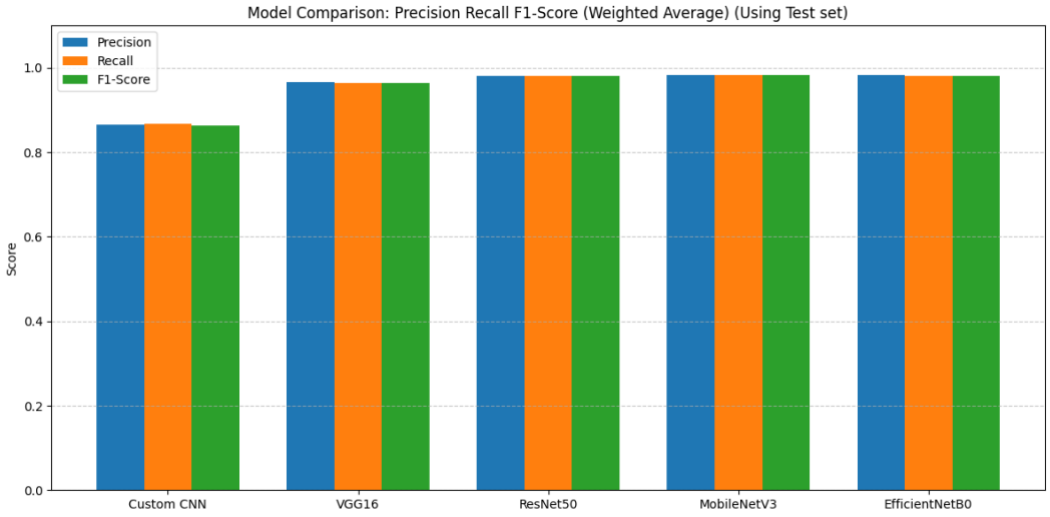

Figure 7 Model Comparison Graph

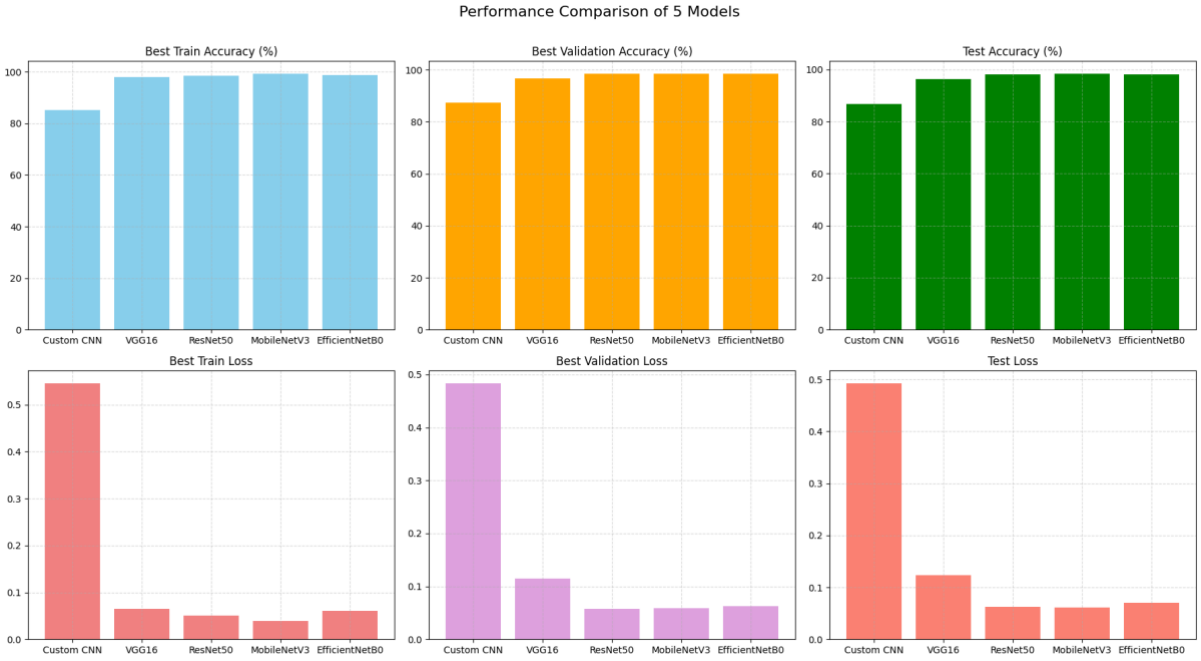

Figure 8 Six Performance Comparison Graphs

Custom CNN performed lower than transfer learning models, with a test accuracy of 86.81%. Test loss was calculated as 0.4922, indicating that the test set's generalisation is limited. F1-score, recall and precision are at an acceptable level. However, it takes a very long time to train a custom CNN.

Transfer learning models were trained in a shorter time than the custom CNN. Although they were trained in a shorter time, they achieved better results. The VGG16 model was trained for approximately 55 minutes and indicates successful performance with a test accuracy of 96.43%. Its F1 Score, recall, and precision were at a very successful rate.

ResNet50, MobileNetV3, and EfficientNet models are the most successful models among all these models. They all have accuracy greater than 98%. They share similar F1 Scores, recall, and precision at a very high rate. The MobileNetV3 model stands out with its training time of 33 minutes, which shows the model is an efficient and accurate alternative.

When all the metrics are considered, transfer learning models are better alternatives than a custom CNN. MobileNetV3 stands out as the better model in terms of both efficiency and overall success. It was concluded that models using transfer learning are more suitable and scalable for real-world applications.

7. Conclusion and Future Work

This project successfully achieved its objective of classifying 226 plant species using artificial intelligence. The results demonstrated that transfer learning models, particularly MobileNetV3, achieved higher accuracy and required less training time than a custom CNN, validating the initial hypothesis. Techniques such as data augmentation, dropout, and L2 regularization proved effective in addressing overfitting and improving generalization.

For future work, creating a more diverse and balanced dataset could further enhance model performance, especially for underrepresented classes. Additionally, deploying the trained model in real-time applications, such as mobile or edge devices, would improve usability and accessibility, making plant identification more practical in outdoor and field environments.

References

Karypidis, E., Mouslech, S. G., Skoulariki, K. and Gazis, A., 2022. Comparison analysis of traditional machine learning and deep learning techniques for data and image classification. arXiv [cs.CV] [online]. Available from: http://arxiv.org/abs/2204.05983.

Wang, P., Fan, E. and Wang, P., 2021. Comparative analysis of image classification algorithms based on traditional machine learning and deep learning. Pattern recognition letters [online], 141, 61–67. Available from: http://dx.doi.org/10.1016/j.patrec.2020.07.042.

Krizhevsky, A., Sutskever, I. and Hinton, G. E., 2017. ImageNet classification with deep convolutional neural networks. Communications of the ACM [online], 60 (6), 84–90. Available from: http://dx.doi.org/10.1145/3065386.

Simonyan, K. and Zisserman, A., 2014. Very deep convolutional networks for large-scale image recognition. arXiv [cs.CV] [online]. Available from: http://arxiv.org/abs/1409.1556.

He, K., Zhang, X., Ren, S. and Sun, J., 2016. Deep residual learning for image recognition. In: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR). Las Vegas, NV, USA, 27–30 June 2016. USA: IEEE, pp.770–778. Available at: https://ieeexplore.ieee.org/document/7780459.

Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M. and Adam, H., 2017. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. arXiv [cs.CV] [online]. Available from: http://arxiv.org/abs/1704.04861.

Ahmed, M.I., Mamun, S.M. and Asif, A.U.Z., 2021. DCNN-based vegetable image classification using transfer learning: A comparative study. In: 2021 5th International Conference on Computer, Communication and Signal Processing (ICCCSP). Chennai, India, 4–6 March 2021. USA: IEEE, pp.235–243. Available at: https://www.researchgate.net/publication/352846889_DCNN-Based_Vegetable_Image_Classification_Using_Transfer_Learning_A_Comparative_Study.

Ahmed, M.I. and Mamun, S.M., 2021. Vegetable Image Dataset [data set]. Kaggle. Available at: https://doi.org/10.34740/KAGGLE/DSV/2965251.

Misrak, A., n.d. *Vegetable Image Dataset*. [online] Kaggle. Available at: https://www.kaggle.com/datasets/misrakahmed/vegetable-image-dataset.

Arfirza, M.I., n.d. *Ornamental Plant Images Dataset* [data set]. Kaggle. Available at: https://www.kaggle.com/datasets/muhammadirvanarfirza/decorative-plant-image-dataset.

Sulistya, Y.I., 2023. Plants Type Datasets [data set]. Kaggle. Available at: https://doi.org/10.34740/KAGGLE/DSV/7170186

Oltean, M., 2017. Fruits-360 [data set]. Kaggle. Available at: https://www.kaggle.com/datasets/moltean/fruits

Appendix

N/A